

MVP Engenharia de Dados

Análise de Despesas Eleitorais — Eleições 2026

Aluno: Marco Antonio Pereira da Silva

Email: marcoantoniopds@icloud.com

GitHub: <https://github.com/marcoantoniopds>

Sprint: Engenharia de Dados

MVP Engenharia de Dados: Análise de Despesas Eleitorais - Eleições 2026.....	1
1. Contexto de Negócios e Perguntas.....	2
2. Coleta e Carregamento dos Dados.....	3
3. Modelagem e Catálogo de Dados.....	6
4. Qualidade de Dados.....	14
5. Pipeline de Dados (ETL).....	19
6. Análise de Dados.....	21
7. Discussão Geral dos Resultados.....	27
8. Pontos de Melhoria.....	27
9. Autoavaliação.....	28

1. Contexto de Negócios e Perguntas

Com a proximidade das eleições de 2026, o projeto busca estruturar informações públicas sobre candidaturas, despesas eleitorais e eleitorado, permitindo analisar de forma mais clara a distribuição dos custos contratados durante o processo eleitoral.

O objetivo é consolidar esses dados por meio de um pipeline de Engenharia de Dados e transformá-los em uma estrutura organizada para análise, permitindo observar a distribuição das despesas entre estados, candidatos, categorias de gastos e eleitorado.

Perguntas de negócio

Com base nesse contexto, foram definidos os seguintes questionamentos:

1. Quais são os 5 estados com maior valor total de despesas eleitorais contratadas em 2026?

1.1. Quais são os 3 candidatos com maior valor total de despesas contratadas em cada um desses 5 estados?

2. Quais são as 3 principais origens das despesas eleitorais contratadas em 2026?

2.1. Quais são as 3 principais descrições de despesas dentro de cada uma das 3 principais origens?

3. Qual é o custo das despesas eleitorais contratadas, por eleitor, em cada estado brasileiro em 2026?

Estrutura dos dados brutos

Foram utilizadas três bases disponibilizadas pelo Tribunal Superior Eleitoral (TSE), em arquivos CSV:

Base de candidaturas

A base de candidaturas contém informações descritivas das candidaturas utilizadas posteriormente nas análises. Entre os principais campos estão `SQ_CANDIDATO`, `NM_CANDIDATO`, `SG_PARTIDO`, `DS_CARGO` e `SG_UF`.

Base de despesas eleitorais contratadas

A base de despesas contém os registros de despesas contratadas e constitui a principal fonte das análises financeiras. Entre os principais campos estão `SQ_DESPESA`, `SQ_CANDIDATO`, `SG_UF`, `DS_ORIGEM_DESPESA`, `DS_DESPESA`, `VR_DESPESA_CONTRATADA` e `DT_DESPESA`.

Base de perfil do eleitorado

A base de eleitorado contém informações agregadas por características geográficas e demográficas. Entre os principais campos estão `DT_GERACAO`, `SG_UF`, `CD_MUNICIPIO`, `NM_MUNICIPIO`, `NR_ZONA`, `DS_GENERO`, `DS_ESTADO_CIVIL`, `DS_FAIXA_ETARIA`, `DS_GRAU_INSTRUCAO`, `DS_COR_RACA` e `QT_ELEITORES`.

Os três arquivos representam snapshots das bases disponíveis na data de execução do projeto. A base de despesas utilizada corresponde à prestação de contas disponível até 24/09/2026.

Licença e uso dos dados

As bases utilizadas neste projeto são disponibilizadas publicamente pelo Tribunal Superior Eleitoral (TSE) por meio de seu Portal de Dados Abertos e estão identificadas no portal sob a licença **Creative Commons Atribuição (CC BY)**.

2. Coleta e Carregamento dos Dados

Como os dados foram carregados?

As três bases selecionadas foram carregadas manualmente no ambiente **Databricks Community Edition**, utilizando um **Volume do Unity Catalog** para armazenamento dos arquivos antes do processamento.

O caminho utilizado no projeto foi:

```
/Volumes/workspace/default/MVP_Engenharia_de_Dados
```

A partir desse local, os arquivos foram lidos diretamente pelo notebook utilizando **PySpark**, respeitando as características de cada fonte, como delimitador, aspas e codificação dos arquivos.

Bases utilizadas e origem

- `consulta_cand_2026_BRASIL.csv` - <https://dadosabertos.tse.jus.br/dataset/candidatos-2026#>
- `despesas_contratadas_candidatos_2026_BRASIL.csv` - <https://dadosabertos.tse.jus.br/dataset/prestacao-de-contas-eleitorais-2026#>
- `perfil_eleitorado_ATUAL.csv` - <https://dadosabertos.tse.jus.br/dataset/eleitorado-atual#>

Devido ao tamanho dos arquivos (+2GB) não será possível disponibilizá-los pelo GITHUB, que limita os uploads em até 25mb.

A leitura foi realizada utilizando arquivos CSV com delimitador `;`. Para as bases de candidaturas e despesas foi utilizado `UTF-8`, enquanto a base de eleitorado foi carregada utilizando `ISO-8859-1`, de acordo com as características identificadas nos arquivos.

Processo de Carga

O processo realizado foi:

Arquivos disponibilizados pelo TSE → Upload para o Volume do Databricks → Leitura dos arquivos com PySpark → DataFrames Bronze → Transformações para a camada Silver

A carga foi realizada manualmente devido ao volume dos arquivos utilizados, especialmente a base de perfil do eleitorado. Os arquivos representam os snapshots disponíveis na data de execução do projeto, extraídos no dia 24/09.

Código de Carregamento Base de Candidaturas (Bronze)

O carregamento dos arquivos foi realizado diretamente a partir do Volume do Databricks. Como exemplo, abaixo está a leitura da [Base de Candidaturas](#)

12:25 PM (42s) 7 Python

```

from pyspark.sql.functions import col, trim, upper

caminho = "/Volumes/workspace/default/MVP_Engenharia_de_Dados"

df_candidatos = (
    spark.read
    .option("header", True)
    .option("delimiter", ";")
    .option("quote", "'")
    .option("encoding", "UTF-8")
    .csv(f"{caminho}/consulta_cand_2026_BRASIL.csv")
)

display(df_candidatos.limit(5))

```

> [See performance \(1\)](#) ✓ Checked for improvements

> df_candidatos: pyspark.sql.connect.dataframe.DataFrame = [DT_GERACAO: string, HH_GERACAO: string ... 48 more fields]

Table +

	DT_GERACAO	HH_GERACAO	ANO_ELEICAO	CD_TIPO_ELEICAO	NM_TIPO_ELEICAO	NR_TURNO	CD_ELEICAO	DS_ELE
1	01/09/2026	19:31:05	2026	2	ELEIÇÃO ORDINÁRIA	1	6259	Eleições
2	01/09/2026	19:31:05	2026	2	ELEIÇÃO ORDINÁRIA	1	6259	Eleições
3	01/09/2026	19:31:05	2026	2	ELEIÇÃO ORDINÁRIA	1	6259	Eleições
4	01/09/2026	19:31:05	2026	2	ELEIÇÃO ORDINÁRIA	1	6259	Eleições
5								

5 rows | 41.733s runtime Refreshed 1h ago

Código de Carregamento Tabela de Despesas Eleitorais (Bronze)

O carregamento dos arquivos foi realizado diretamente a partir do Volume do Databricks. Como exemplo, abaixo está a leitura da [Base de despesas eleitorais contratadas](#)

12:25 PM (3s) 11

```

df_despesas = (
    spark.read
    .option("header", True)
    .option("delimiter", ";")
    .option("quote", "'")
    .option("encoding", "UTF-8")
    .csv(f"{caminho}/despesas_contratadas_candidatos_2026_BRASIL.csv")
)

display(df_despesas.limit(5))

```

> [See performance \(1\)](#) ✓ Checked for improvements

> df_despesas: pyspark.sql.connect.dataframe.DataFrame = [DT_GERACAO: string, HH_GERACAO: string ... 51 more fields]

Table +

	DT_GERACAO	HH_GERACAO	AA_ELEICAO	CD_TIPO_ELEICAO	NM_TIPO_ELEICAO	CD_ELEICAO	DS_ELEICAO	
1	24/09/2026	04:05:37	2026	2	Ordinária	6259	Eleições Gerais Estaduais 20...	04
2	24/09/2026	04:05:37	2026	2	Ordinária	6259	Eleições Gerais Estaduais 20...	04
3	24/09/2026	04:05:37	2026	2	Ordinária	6259	Eleições Gerais Estaduais 20...	04
4	24/09/2026	04:05:37	2026	2	Ordinária	6259	Eleições Gerais Estaduais 20...	04
5								

5 rows | 2.847s runtime Refreshed 2h ago

Código de Carregamento Tabela de Perfil Eleitoral (Bronze)

O carregamento dos arquivos foi realizado diretamente a partir do Volume do Databricks. Como exemplo, abaixo está a leitura da [Base de perfil do eleitorado](#)

12:25 PM (3s) 15

```
caminho = "/Volumes/workspace/default/MVP_Engenharia_de_Dados"

df_eleitorado = (
    spark.read
    .option("header", True)
    .option("delimiter", ";")
    .option("quote", "'")
    .option("encoding", "ISO-8859-1")
    .csv(f"{caminho}/perfil_eleitorado_ATUAL.csv")
)

display(df_eleitorado.limit(5))
```

> [See performance \(1\)](#) ✓ Checked for improvements

> df_eleitorado: pyspark.sql.connect.dataframe.DataFrame = [DT_GERACAO: string, HH_GERACAO: string ... 26 more fields]

	DT_GERACAO	HH_GERACAO	ANO_ELEICAO	SG_UF	CD_MUNICIPIO	NM_MUNICIPIO	NR_ZONA	CD_GENERO
1	03/08/2026	11:37:53	9999	AC	1007	BUJARI	9	2
2	03/08/2026	11:37:53	9999	AC	1007	BUJARI	9	2
3	03/08/2026	11:37:53	9999	AC	1007	BUJARI	9	2
4	03/08/2026	11:37:53	9999	AC	1007	BUJARI	9	2
5								

5 rows | 3.064s runtime Refreshed 2h ago

3. Modelagem e Catálogo de Dados

Preparação dos dados - Base de Candidaturas (Silver)

Na [Base de Candidaturas](#), após a ingestão dos dados na camada Bronze, foram realizados os seguintes processos de padronização:

- Conversão do identificador do candidato para o tipo numérico inteiro.
- Remoção de espaços excedentes nos campos textuais.
- Padronização dos campos de partido e estado para letras maiúsculas.
- Renomeação dos campos para uma nomenclatura padronizada na camada Silver.
- Estruturação dos dados com uma linha por candidato, utilizando `ID_CANDIDATO` como identificador da candidatura.

12:25 PM (2s) 9

```
df_candidatos_silver = df_candidatos.select(
    col("SQ_CANDIDATO").cast("long").alias("ID_CANDIDATO"),
    trim(col("NM_CANDIDATO")).alias("CANDIDATO"),
    upper(trim(col("SG_PARTIDO"))).alias("PARTIDO"),
    trim(col("DS_CARGO")).alias("CARGO"),
    upper(trim(col("SG_UF"))).alias("ESTADO")
)

display(df_candidatos_silver.limit(10))
```

See performance (1) ✓ Checked for improvements

df_candidatos_silver: pyspark.sql.connect.dataframe.DataFrame = [ID_CANDIDATO: long, CANDIDATO: string ... 3 more fields]

	ID_CANDIDATO	CANDIDATO	PARTIDO	CARGO	ESTADO
1	260002551366	HENRIQUE ALVES DA ROCHA	PL	SENADOR	SE
2	260002547290	ANDRE LUIS DANTAS FERREIRA	UNIÃO	SENADOR	SE
3	30002552017	MARCOS JOSE REATEGUI DE SOUZA	DC	GOVERNADOR	AP
4	250002552369	SONIA FRANCINE GASPAR MARMO	CIDADANIA	SENADOR	SP
5	70002552965	PAULA MORENO PARO BELMONTE	PSDB	GOVERNADOR	DF
6	120002552201	DANIEL LEMES VASQUES	PCO	GOVERNADOR	MS
7	40002537344	FRANCISCO PLINIO VALERIO TOMAZ	PSDB	SENADOR	AM
8	40002551307	FRANCISCO MORAIARE ROMANO	REDE	DEPUTADO ESTADUAL	AM
9	250002554075	RICARDO AUGUSTO MANGUE SCHIAVETTO	MISSO	SENADOR	SP
10	120002549681	RENATO WANDERLEY GOMES	DC	GOVERNADOR	MS

10 rows 2.184s runtime Refreshed 2h ago

Preparação dos dados - Despesas Eleitorais (Silver)

Na [Base de despesas eleitorais contratadas](#), após a ingestão dos dados na camada Bronze, foram realizados os seguintes processos de padronização:

- Conversão dos identificadores de despesa e candidato para o tipo numérico inteiro.
- Padronização do estado e remoção de espaços excedentes nos campos textuais.
- Conversão dos valores de despesas para o formato numérico decimal.
- Conversão da data da despesa para o formato de data.
- Renomeação e padronização dos campos para a estrutura da **FACT_DESPESA**.
- Manutenção dos registros da fonte, sem exclusão de dados durante a transformação para a camada Silver.

12:25 PM (14) 13

```

from pyspark.sql.functions import col, trim, upper, to_date, regexp_replace

df_despesas_silver = df_despesas.select(
    col("SQ_DESPESA")
    .cast("long")
    .alias("ID_DESPESA"),
    col("SQ_CANDIDATO")
    .cast("long")
    .alias("ID_CANDIDATO"),
    upper(trim(col("SG_UF")))
    .alias("ESTADO"),
    trim(col("DS_ORIGEM_DESPESA"))
    .alias("CATEGORIA_DESPESA"),
    trim(col("DS_DESPESA"))
    .alias("DESPESA_DETALHADA"),
    regexp_replace(
        regexp_replace(
            trim(col("VR_DESPESA_CONTRATADA")),
            "\\.+",
            ""
        ),
        "\.+",
        ""
    )
    .cast("decimal(18,2)")
    .alias("VALOR_DESPESA"),
    to_date(
        trim(col("DT_DESPESA")),
        "dd/MM/yyyy"
    )
    .alias("DATA_DESPESA")
)

display(df_despesas_silver.limit(10))

```

See performance (1) ✓ Checked for improvements

df_despesas_silver: pyspark.sql.connect.dataframe.DataFrame = [ID_DESPESA: long, ID_CANDIDATO: long ... 5 more fields]

	ID_DESPESA	ID_CANDIDATO	ESTADO	CATEGORIA_DESPESA	DESPESA_DETALHADA	VALOR_DESPESA
1	74870484	70002550992	DF	Despesas com pessoal	camiseta	20
2	74870484	70002550992	DF	Despesas com pessoal	bandeira	4
3	74870484	70002550992	DF	Publicidade em material impresso	placardinha - 101R211	4

Preparação dos dados - Tabela de Perfil Eleitoral (Silver)

Na [Base de perfil do eleitorado](#), após a ingestão dos dados na camada Bronze, foram realizados os seguintes processos de padronização:

- Conversão dos códigos de município e zona eleitoral para tipos numéricos.
- Padronização do estado para letras maiúsculas e remoção de espaços excedentes.
- Conversão da data de geração para o formato de data.
- Remoção de espaços excedentes nos campos textuais relacionados ao perfil do eleitorado.
- Conversão das quantidades de eleitores para tipos numéricos inteiros.
- Renomeação dos campos para uma nomenclatura padronizada na camada Silver.

12:25 PM (1s) 17

```
df_eleitorado_silver = df_eleitorado.select(
    to_date(col("DT_GERACAO"), "dd/MM/yyyy").alias("DATA_GERACAO"),
    upper(trim(col("SG_UF"))).alias("ESTADO"),
    col("CD_MUNICIPIO").cast("int").alias("CODIGO_MUNICIPIO"),
    trim(col("NM_MUNICIPIO")).alias("MUNICIPIO"),
    col("NR_ZONA").cast("int").alias("ZONA"),
    trim(col("DS_GENERO")).alias("GENERO"),
    trim(col("DS_ESTADO_CIVIL")).alias("ESTADO_CIVIL"),
    trim(col("DS_FAIXA_ETARIA")).alias("FAIXA_ETARIA"),
    trim(col("DS_GRAU_INSTRUCAO")).alias("GRAU_INSTRUCAO"),
    trim(col("DS_COR_RACA")).alias("COR_RACA"),
    trim(col("TP_OBRIGATORIEDADE_VOTO")).alias("OBRIGATORIEDADE_VOTO"),
    col("QT_ELEITORES").cast("long").alias("ELEITORES"),
    col("QT_ELEITORES_BIOMETRIA").cast("long").alias("ELEITORES_BIOMETRIA"),
    col("QT_ELEITORES_DEFICIENCIA").cast("long").alias("ELEITORES_DEFICIENCIA"),
    col("QT_ELEITORES_NOME_SOCIAL").cast("long").alias("ELEITORES_NOME_SOCIAL")
)

display(df_eleitorado_silver.limit(10))
```

> See performance (1) ✓ Checked for improvements

> df_eleitorado_silver: pyspark.sql.connect.dataframe.DataFrame = [DATA_GERACAO: date, ESTADO: string ... 13 more fields]

	DATA_GERACAO	ESTADO	CODIGO_MUNICIPIO	MUNICIPIO	ZONA	GENERO	ESTADO_CIVIL	FAIXA_ETARIA	GRAU_INSTRUCAO
1	2026-08-03	AC	1007	BUJARI	9	MASCULINO	SOLTEIRO	16 anos	ENSINO FUNDAMENTAL INC
2	2026-08-03	AC	1007	BUJARI	9	MASCULINO	SOLTEIRO	16 anos	ENSINO FUNDAMENTAL COF
3	2026-08-03	AC	1007	BUJARI	9	MASCULINO	SOLTEIRO	21 a 24 anos	ANALFABETO
4	2026-08-03	AC	1007	BUJARI	9	MASCULINO	SOLTEIRO	35 a 39 anos	ANALFABETO
5	2026-08-03	AC	1007	BUJARI	9	MASCULINO	SOLTEIRO	45 a 49 anos	ANALFABETO
6	2026-08-03	AC	1007	BUJARI	9	MASCULINO	SOLTEIRO	35 a 39 anos	ANALFABETO
7	2026-08-03	AC	1007	BUJARI	9	MASCULINO	SOLTEIRO	55 a 59 anos	ANALFABETO
8	2026-08-03	AC	1007	BUJARI	9	MASCULINO	SOLTEIRO	70 a 74 anos	LÊ E ESCRIVE

Modelagem dos dados

A partir das estruturas preparadas na camada Silver, foi definido um **esquema estrela (Star Schema)** com a **FACT_DESPESA** como tabela fato central e quatro dimensões relacionadas às principais análises do projeto.

A escolha desse modelo permite organizar as informações de despesas por diferentes perspectivas analíticas, como candidato, estado, categoria de despesa e período.

Estrutura do modelo

FACT_DESPESA

Tabela fato central, contendo uma linha por registro de despesa disponibilizado pela fonte.

Armazena a medida **VALOR_DESPESA** e as chaves de relacionamento com as dimensões.

DIM_CANDIDATO

Contém os atributos descritivos das candidaturas, com uma linha por candidato e `ID_CANDIDATO` como chave primária.

DIM_ESTADO

Contém uma linha por estado e o `TOTAL_ELEITORES`, obtido pela agregação da base de eleitorado por unidade federativa.

DIM_CATEGORIA_DESPESA

Contém uma linha por combinação distinta de categoria e descrição de despesa, permitindo detalhar a origem e o tipo dos gastos.

DIM_DATA

Contém uma linha por data de despesa, com atributos temporais derivados como ano, mês, dia e trimestre.

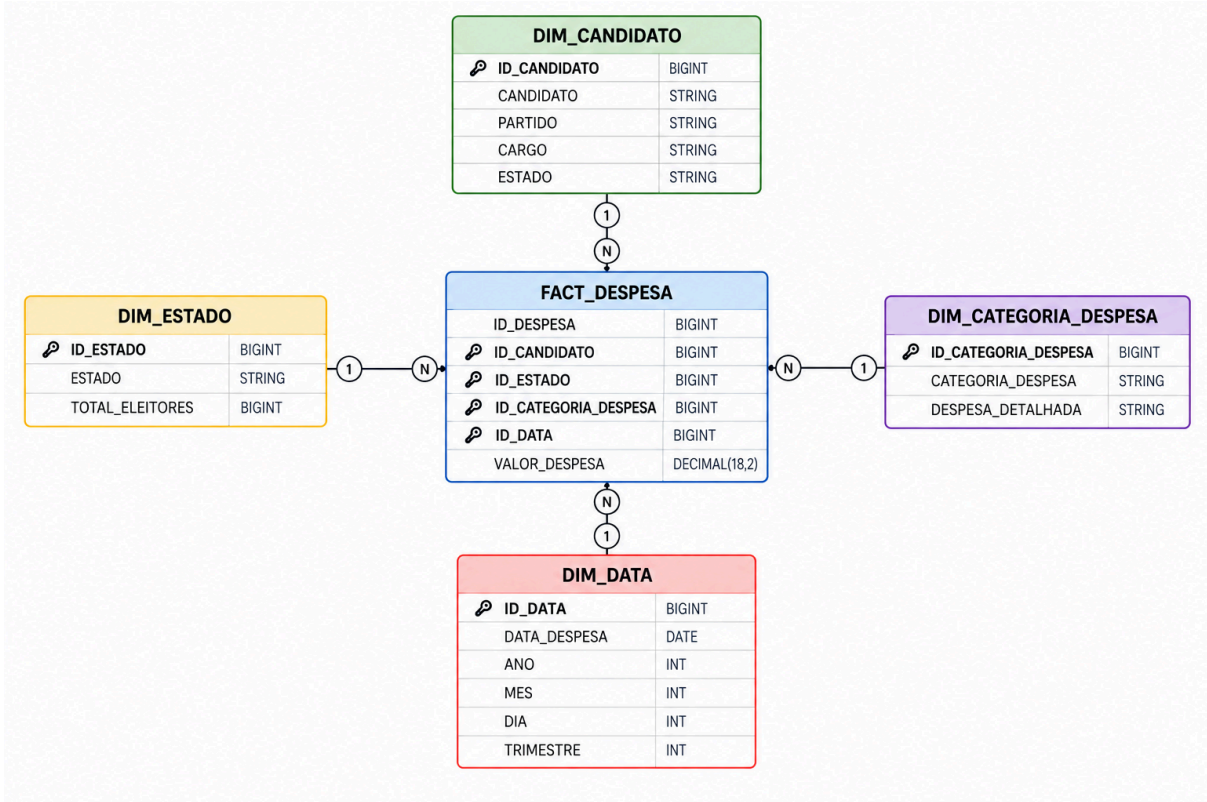
Relacionamentos

A `FACT_DESPESA` possui relacionamento direto com as quatro dimensões:

- `DIM_CANDIDATO (1) → (N) FACT_DESPESA`
- `DIM_ESTADO (1) → (N) FACT_DESPESA`
- `DIM_CATEGORIA_DESPESA (1) → (N) FACT_DESPESA`
- `DIM_DATA (1) → (N) FACT_DESPESA`

A base detalhada de eleitorado permanece na camada Silver e seus dados são agregados por `ESTADO` para composição da `DIM_ESTADO`, evitando um relacionamento direto entre a granularidade do eleitorado e a granularidade das despesas.

Diagrama do modelo



Catálogo de Dados

O catálogo de dados foi desenvolvido para documentar a estrutura das tabelas modeladas, apresentando os campos de origem, os campos modelados, tipos de dados, descrições, domínios ou faixas esperadas, linhagem e papel de cada atributo no modelo.

DIM_CANDIDATO

Campo original TSE	Campo modelado	Tipo	Descrição	Domínio / Faixa esperada	Linhagem	Papel
SQ_CANDIDATO	ID_CANDIDATO	BIGINT	Identificador único da candidatura, gerado na modelagem.	Inteiros positivos. Deve ser único por candidato.	Derivado da coluna SQ_CANDIDATO da base de candidatos (bronze/silver), com cast para BIGINT.	PK
NM_CANDIDATO	CANDIDATO	STRING	Nome do candidato.	Texto não nulo. Pode conter acentos e caracteres especiais.	Derivado da coluna NM_CANDIDATO, com aplicação de trim para remoção de espaços excedentes.	Atributo
SG_PARTIDO	PARTIDO	STRING	Sigla do partido político do candidato.	Siglas válidas de partidos registrados no TSE (ex.: PL, PT, MDB, PSDB, etc.).	Derivado da coluna SG_PARTIDO, com aplicação de trim e upper (padronização para maiúsculo).	Atributo
DS_CARGO	CARGO	STRING	Cargo ao qual o candidato concorre.	Valores definidos pelo TSE (ex.: Deputado Federal, Senador, Governador, etc.).	Derivado da coluna DS_CARGO, com aplicação de trim para remoção de espaços excedentes.	Atributo
SG_UF	ESTADO	STRING	Sigla da unidade federativa do candidato.	Siglas válidas dos estados brasileiros (AC, AL, AP, AM, BA, CE, DF, ES, GO, MA, MT, MS, MG, PA, PB, PR, PE, PI, RJ, RN, RS, RO, RR, SC, SP, SE, TO).	Derivado da coluna SG_UF, com aplicação de trim e upper (padronização para maiúsculo).	Atributo

FACT_DESPESA

Campo original TSE	Campo modelado	Tipo	Descrição	Domínio / Faixa esperada	Linhagem	Papel
SQ_DESPESA	ID_DESPESA	BIGINT	Identificador da despesa conforme disponibilizado pela fonte.	Inteiro; pode assumir -1 em registros anômalos. Não é necessariamente único.	Derivado de SQ_DESPESA da Silver por cast long; mantido na FACT_DESPESA.	Atributo
SQ_CANDIDATO	ID_CANDIDATO	BIGINT	Chave de relacionamento com a dimensão de candidatos.	Inteiros; -1 para candidato não identificado.	Derivado de SQ_CANDIDATO na Silver e relacionado à DIM_CANDIDATO; quando não há correspondência, recebe -1.	FK
SG_UF	ID_ESTADO	BIGINT	Chave de relacionamento com a dimensão de estados.	Inteiros positivos correspondentes às UFs modeladas.	SG_UF da Silver é relacionado ao ESTADO da DIM_ESTADO para obter ID_ESTADO.	FK
DS_ORIGEM_DESPESA	ID_CATEGORIA_DESPESA	BIGINT	Chave de relacionamento com a dimensão de categorias de despesa.	Inteiros positivos correspondentes às combinações de categoria e descrição modeladas.	DS_ORIGEM_DESPESA + DS_DESPESA da Silver são relacionados à DIM_CATEGORIA_DESPESA para obter ID_CATEGORIA_DESPESA.	FK
DT_DESPESA	ID_DATA	BIGINT	Chave de relacionamento com a dimensão de data da despesa.	0 para data desconhecida; valores válidos no formato yyyyMMdd.	DT_DESPESA convertido para DATA_DESPESA na Silver e relacionado à DIM_DATA; datas ausentes recebem ID_DATA = 0.	FK
VR_DESPESA_CONTRATADA	VALOR_DESPESA	DECIMAL(18,2)	Valor da despesa contratada em reais.	Valores monetários em reais; registros anômalos da fonte podem apresentar 0,00.	VR_DESPESA_CONTRATADA da Silver após padronização do separador decimal e cast para decimal(18,2), mantido na FACT_DESPESA.	Medida

DIM_ESTADO

Campo original TSE	Campo modelado	Tipo	Descrição	Domínio / Faixa esperada	Linhagem	Papel
—	ID_ESTADO	BIGINT	Identificador único do estado, gerado na modelagem.	Inteiros positivos (1 até N).	Gerado com row_number() sobre os estados distintos da tabela de despesas, ordenados por ESTADO.	PK
SG_UF	ESTADO	STRING	Sigla da unidade federativa.	Siglas das unidades federativas presentes na base de despesas.	Derivado de df_despesas_silver.ESTADO, após trim e upper.	Atributo
QT_ELEITORES	TOTAL_ELEITORES	BIGINT	Total de eleitores obtido pela agregação do eleitorado por estado.	Inteiros maiores ou iguais a 0.	SUM(ELEITORES) da df_eleitorado_silver agrupada por ESTADO.	Medida

DIM_CATEGORA_DESPESA

Campo original TSE	Campo modelado	Tipo	Descrição	Domínio / Faixa esperada	Linhagem	Papel
—	ID_CATEGORIA_DESPESA	BIGINT	Identificador único da categoria de despesa, gerado na modelagem.	Inteiros positivos (1 até N).	Gerado a partir da coluna DS_ORIGEM_DESPESA da tabela de despesas (valores distintos).	PK
DS_ORIGEM_DESPESA	CATEGORIA_DESPESA	STRING	Categoria de despesa conforme a descrição de origem no TSE.	Valores textuais conforme dicionário do TSE (ex.: Bens e serviços, Publicidade, etc.).	Diretamente da coluna DS_ORIGEM_DESPESA da tabela de despesas (padronizada com trim/upper quando aplicável).	Atributo
DS_DESPESA	DESPESA_DETALHADA	STRING	Descrição detalhada da despesa.	Valores textuais conforme dicionário do TSE. Pode conter registros nulos ou valores como "#NULO" na fonte.	Diretamente da coluna DS_DESPESA da tabela de despesas (padronizada com trim quando aplicável).	Atributo

DIM_DATA

Campo original TSE	Campo modelado	Tipo	Descrição	Domínio / Faixa esperada	Linhagem	Papel
—	ID_DATA	BIGINT	Identificador da data, gerado na modelagem a partir de DATA_DESPESA no formato yyyyMMdd. O valor 0 é reservado para data desconhecida.	0 para data desconhecida; para datas válidas, formato yyyyMMdd.	Gerado com date_format(DATA_DESPESA, yyyyMMdd) e cast long.	PK
DT_DESPESA	DATA_DESPESA	DATE	Data da despesa contratada.	Datas válidas de 2026 e NULL na origem quando aplicável.	Conversão de DT_DESPESA da Silver com to_date(dd/MM/yyyy).	Atributo
DT_DESPESA	ANO	INT	Ano da data da despesa.	2026	Derivado de DATA_DESPESA com year().	Atributo
DT_DESPESA	MES	INT	Mês da data da despesa.	1 a 12	Derivado de DATA_DESPESA com month().	Atributo
DT_DESPESA	DIA	INT	Dia da data da despesa.	1 a 31	Derivado de DATA_DESPESA com dayofmonth().	Atributo
DT_DESPESA	TRIMESTRE	INT	Trimestre da data da despesa.	1 a 4	Derivado de DATA_DESPESA com quarter().	Atributo

Sistema Catálogo Dentro do Databricks

As tabelas da camada Gold foram persistidas no ambiente Databricks e disponibilizadas no Unity Catalog, conforme demonstrado na imagem abaixo. A evidência apresenta as estruturas criadas no schema default, incluindo as dimensões, a tabela fato e a tabela analítica utilizada nas consultas de negócio.

Catalog Explorer > workspace >

default

U:

Overview

Details

Permissions

Policies

Description

Default schema (auto-created)

Filter tables

Tables 29

Volumes 2

Models 0

Functions 0

Secrets 0

Services 0

Skills 0

Connections 0

Sort: Created at

Name	Owner	Created at	Popularity	Comment
gold_despesa_analitica	marcoantonioacademic@g...	Sep 26, 2026, 12:30 PM	...	
fact_despesa	marcoantonioacademic@g...	Sep 26, 2026, 12:30 PM	...	
dim_data	marcoantonioacademic@g...	Sep 26, 2026, 12:30 PM	...	
dim_categoria_despesa	marcoantonioacademic@g...	Sep 26, 2026, 12:30 PM	...	
dim_estado	marcoantonioacademic@g...	Sep 26, 2026, 12:30 PM	...	
dim_candidato	marcoantonioacademic@g...	Sep 26, 2026, 12:29 PM	...	

4. Qualidade de Dados

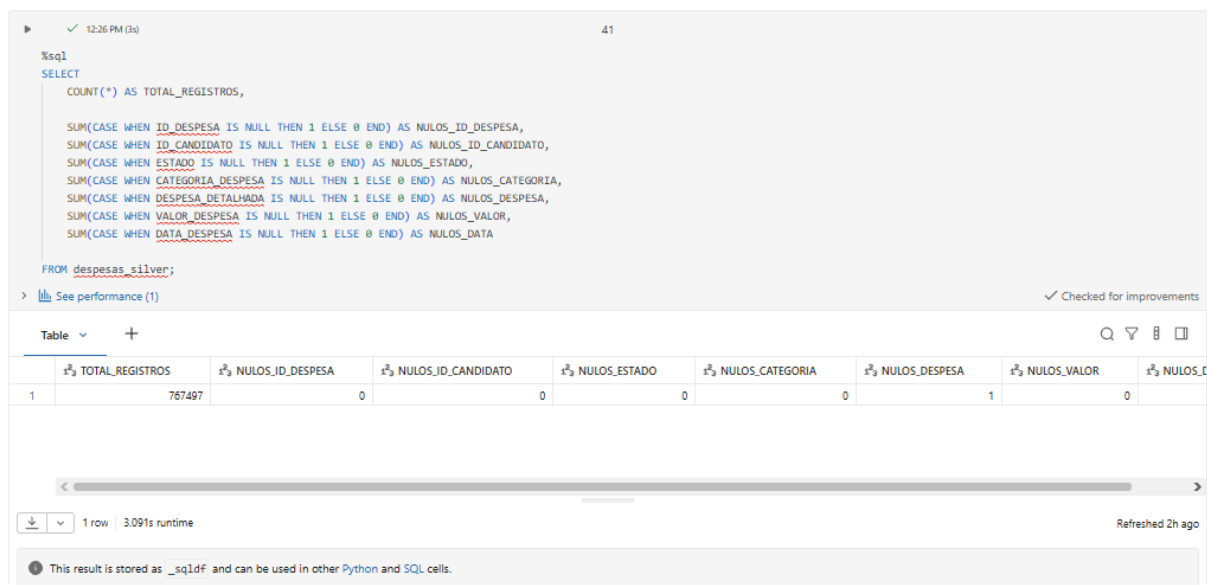
A etapa de qualidade apresentada a seguir ocorre após a preparação da camada Silver e antes da consolidação definitiva das estruturas Gold, sendo detalhada separadamente devido à sua importância para a confiabilidade do modelo e das análises.

Foram avaliados aspectos de **completude**, **unicidade**, **consistência**, **integridade referencial**, **valores negativos** e **registros anômalos**, considerando a granularidade e a finalidade de cada tabela.

Qualidade da tabela de despesas

A Tabela de Despesas foi a base que apresentou a maior quantidade de ocorrências que exigiram investigação.

Foram identificados **767.497 registros** na camada Silver - [Preparação dos dados - Despesas Eleitorais \(Silver\)](#)



```
%sql
SELECT
    COUNT(*) AS TOTAL_REGISTROS,
    SUM(CASE WHEN ID_DESPESA IS NULL THEN 1 ELSE 0 END) AS NULOS_ID_DESPESA,
    SUM(CASE WHEN ID_CANDIDATO IS NULL THEN 1 ELSE 0 END) AS NULOS_ID_CANDIDATO,
    SUM(CASE WHEN ESTADO IS NULL THEN 1 ELSE 0 END) AS NULOS_ESTADO,
    SUM(CASE WHEN CATEGORIA_DESPESA IS NULL THEN 1 ELSE 0 END) AS NULOS_CATEGORIA,
    SUM(CASE WHEN DESPESA_DETALHADA IS NULL THEN 1 ELSE 0 END) AS NULOS_DESPESA,
    SUM(CASE WHEN VALOR_DESPESA IS NULL THEN 1 ELSE 0 END) AS NULOS_VALOR,
    SUM(CASE WHEN DATA_DESPESA IS NULL THEN 1 ELSE 0 END) AS NULOS_DATA
FROM despesas_silver;
```

	TOTAL_REGISTROS	NULOS_ID_DESPESA	NULOS_ID_CANDIDATO	NULOS_ESTADO	NULOS_CATEGORIA	NULOS_DESPESA	NULOS_VALOR	NULOS_DATA
1	767497	0	0	0	0	1	0	0

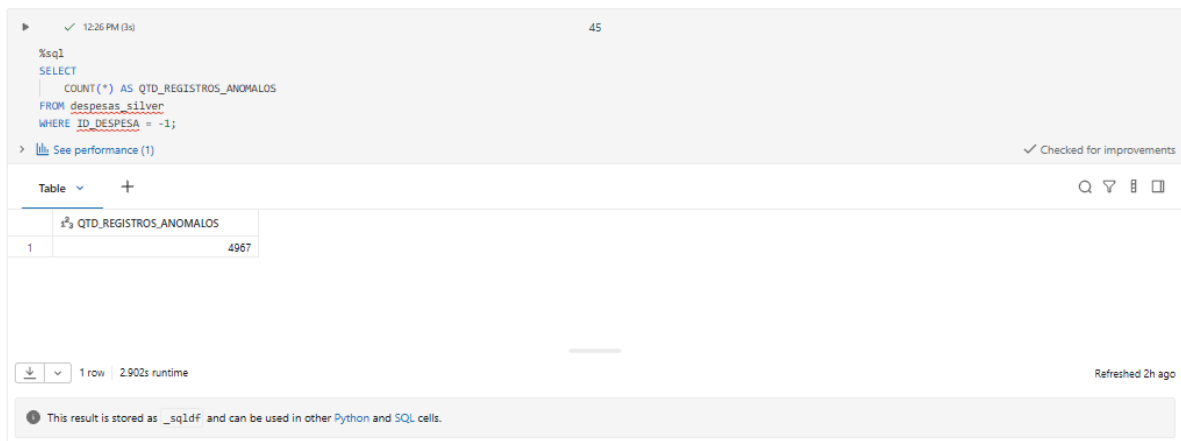
1 row | 3.091s runtime | Refreshed 2h ago

This result is stored as _sqlidf and can be used in other Python and SQL cells.

Registros anômalos

Foram identificados **4.967 registros** com o seguinte padrão:

- `ID_DESPESA = -1;`
- `DATA_DESPESA = NULL;`
- `VALOR_DESPESA = 0,00;`
- `DESPESA_DETALHADA = #NULO.`



```
SELECT
  COUNT(*) AS QTD_REGISTROS_ANOMALOS
FROM despesas_silver
WHERE ID_DESPESA = -1;
```

	QTD_REGISTROS_ANOMALOS
1	4967

1 row · 2.902s runtime · Refreshed 2h ago

This result is stored as `_sqlidf` and can be used in other Python and SQL cells.

Esses registros foram **mantidos na camada Silver**, evitando a exclusão de informações presentes na fonte original e preservando a rastreabilidade dos dados.

Na camada Gold, os registros sem data válida foram direcionados para o identificador `ID_DATA = 0`, representando uma data desconhecida.

Unicidade do identificador de despesa

A análise identificou **37.772 ID_DESPESA com múltiplas ocorrências**, desconsiderando os registros com `ID_DESPESA = -1`.

A investigação demonstrou que um mesmo identificador pode estar associado a diferentes registros, inclusive com descrições e valores distintos. Dessa forma, `ID_DESPESA` não foi considerado uma chave primária simples no nível de granularidade adotado para a `FACT_DESPESA`.

12:26 PM (3s) 51

```
%sql
SELECT
  COUNT(*) AS IDS_DUPLICADOS
FROM (
  SELECT
    ID_DESPESA
  FROM despesas_silver
  WHERE ID_DESPESA <> -1
  GROUP BY ID_DESPESA
  HAVING COUNT(*) > 1
);
```

> See performance (1) ✓ Checked for improvements

IDS_DUPLICADOS	
1	37772

1 row | 2.508s runtime Refreshed 3h ago

This result is stored as `_sqlidf` and can be used in other Python and SQL cells.

Integridade referencial

Foram identificados **168 registros de despesas sem correspondência na tabela de candidatos**.

Como tratamento, foi criado um registro especial na `DIM_CANDIDATO` com:

`ID_CANDIDATO = -1`

e descrição:

`CANDIDATO NÃO IDENTIFICADO`

Na construção da `FACT_DESPESA`, os registros sem correspondência foram associados a esse identificador, evitando a perda das despesas e preservando a integridade do modelo dimensional.

12:26 PM (3s) 47

```
%sql
SELECT
  COUNT(*) AS DESPESAS_SEM_CANDIDATO
FROM despesas_silver d
LEFT JOIN candidatos_silver c
  ON d.ID_CANDIDATO = c.ID_CANDIDATO
WHERE c.ID_CANDIDATO IS NULL;
```

See performance (1) ✓ Checked for improvements

	x ₃ DESPESAS_SEM_CANDIDATO
1	168

1 row | 3.226s runtime Refreshed 3h ago

This result is stored as _sqlidf and can be used in other Python and SQL cells.

Qualidade da tabela de candidatos

A [Preparação dos dados - Base de Candidaturas \(Silver\)](#) apresentou **20.836 registros e 20.836 ID_CANDIDATO distintos**.

Não foram identificados valores nulos nos campos analisados nem duplicidades da chave **ID_CANDIDATO**.

Dessa forma, a estrutura foi considerada adequada para utilização como dimensão de candidatos no modelo.

12:26 PM (2s) 61

```
%sql
SELECT
  COUNT(*) AS TOTAL_REGISTROS,
  COUNT(DISTINCT ID_CANDIDATO) AS IDS_DISTINTOS,
  COUNT(*) - COUNT(DISTINCT ID_CANDIDATO) AS POSSIVEIS_DUPLICIDADES
FROM candidatos_silver;
```

See performance (1) ✓ Checked for improvements

	x ₃ TOTAL_REGISTROS	x ₃ IDS_DISTINTOS	x ₃ POSSIVEIS_DUPLICIDADES
1	20836	20836	0

1 row | 1.795s runtime Refreshed 3h ago

This result is stored as _sqlidf and can be used in other Python and SQL cells.

Qualidade da tabela de eleitorado

A [Preparação dos dados - Tabela de Perfil Eleitoral \(Silver\)](#) apresentou **1.105.719 registros**.



```
%sql
SELECT
  COUNT(*) AS TOTAL_REGISTROS
FROM eleitorado_silver;
```

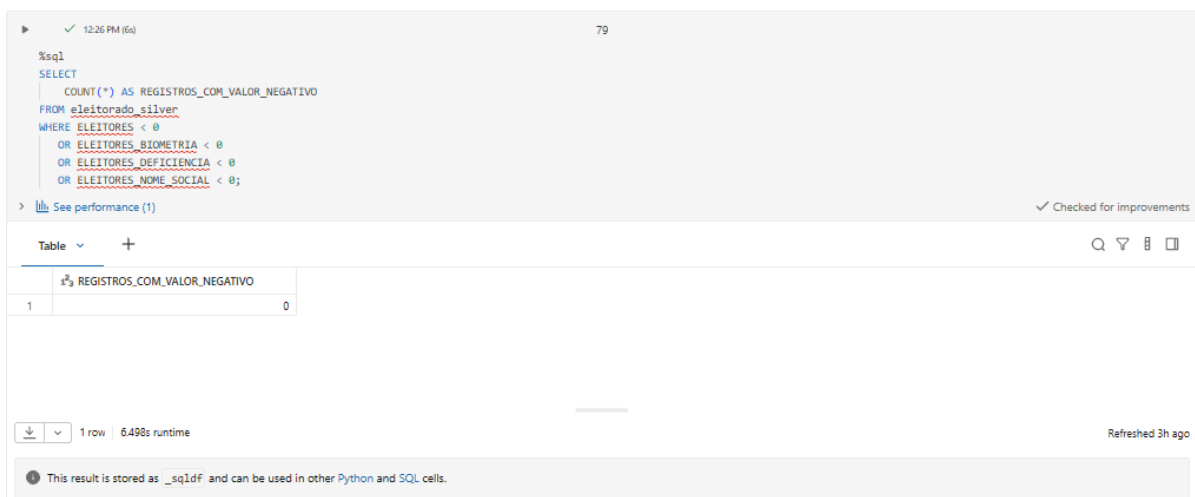
TOTAL_REGISTROS
11057189

1 row | 4.376s runtime

Refreshed 3h ago

This result is stored as `_sqlidf` and can be used in other Python and SQL cells.

Não foram identificados valores nulos nos campos analisados nem valores negativos nas medidas de eleitores.



```
%sql
SELECT
  COUNT(*) AS REGISTROS_COM_VALOR_NEGATIVO
FROM eleitorado_silver
WHERE ELETORES < 0
OR ELETORES_BIOMETRIA < 0
OR ELETORES_DEFICIENCIA < 0
OR ELETORES_NOME_SOCIAL < 0;
```

REGISTROS_COM_VALOR_NEGATIVO
0

1 row | 6.498s runtime

Refreshed 3h ago

This result is stored as `_sqlidf` and can be used in other Python and SQL cells.

A análise da granularidade mostrou que a base é multidimensional, combinando informações de localização e características demográficas. Por esse motivo, não foi definida uma chave primária simples para essa estrutura.

Para utilização na camada Gold, os dados foram agregados por `ESTADO` para composição da `DIM_ESTADO`.

Tratamento dos problemas identificados

Os principais tratamentos realizados durante o projeto foram:

- preservação dos registros anômalos na camada Silver;
- não utilização de `ID_DESPESA` como chave primária simples;

- criação de um registro de candidato não identificado para preservar registros sem correspondência;
- utilização de `ID_DATA = 0` para despesas sem data válida;
- agregação do eleitorado por estado para compatibilizar sua granularidade com as análises de despesas.

Os testes realizados, os resultados encontrados e as consultas utilizadas para investigação estão apresentados no notebook principal do projeto.

5. Pipeline de Dados (ETL)

O pipeline foi desenvolvido integralmente em um único notebook no ambiente Databricks Community Edition, concentrando as etapas de ingestão, transformação, validação, modelagem e disponibilização dos dados para análise.

O processo segue uma estrutura sequencial de ETL, iniciando na leitura dos arquivos disponibilizados pelo TSE, passando pela preparação dos dados na camada Silver e pela construção das estruturas da camada Gold.

Na etapa de Extract, os arquivos foram lidos a partir do Volume do Databricks utilizando PySpark, conforme apresentado no [Processo de Carga](#).

Na etapa de Transform, foram realizados tratamentos de tipos, padronização de campos, tratamento de datas e valores monetários, além da construção das dimensões e da tabela fato do esquema estrela, conforme apresentado no tópico [3. Modelagem e Catálogo de Dados](#).

Por fim, na etapa de Load, as estruturas da camada Gold e a tabela analítica `gold_despesa_analitica` foram persistidas como tabelas no ambiente Databricks, permitindo sua disponibilização no Unity Catalog e posterior utilização nas análises SQL.

```
12:26 PM (1m) 105
# Persistência das dimensões, tabela fato e tabela analítica
df_dim_candidato.write.mode("overwrite").saveAsTable("default.dim_candidato")
df_dim_estado.write.mode("overwrite").saveAsTable("default.dim_estado")
df_dim_categoria_despesa.write.mode("overwrite").saveAsTable("default.dim_categoria_despesa")
df_dim_data.write.mode("overwrite").saveAsTable("default.dim_data")
df_fact_despesa.write.mode("overwrite").saveAsTable("default.fact_despesa")
df_gold_flat.write.mode("overwrite").saveAsTable("default.gold_despesa_analitica")
See performance (5) ✓ Checked for improvements
```

Description

Default schema (auto-created)

Filter tables	Tables 29	Volumes 2	Models 0	Functions 0	Secrets 0	Services 0	Skills 0	Connections 0
Sort: Created at								
Name	Owner	Created at	Popularity	Comment				
gold_despesa_analitica	marcoantonioacademic@g...	Sep 26, 2026, 12:30 PM						
fact_despesa	marcoantonioacademic@g...	Sep 26, 2026, 12:30 PM						
dim_data	marcoantonioacademic@g...	Sep 26, 2026, 12:30 PM						
dim_categoria_despesa	marcoantonioacademic@g...	Sep 26, 2026, 12:30 PM						
dim_estado	marcoantonioacademic@g...	Sep 26, 2026, 12:30 PM						
dim_candidato	marcoantonioacademic@g...	Sep 26, 2026, 12:29 PM						

Temos acesso a essas tabelas dentro do catálogo.

O fluxo geral do processo pode ser representado por:

Fonte TSE → Bronze → Silver → Qualidade → Gold – Esquema Estrela → Gold Flat → Análises SQL

6. Análise de Dados

A etapa de análise foi realizada a partir da tabela analítica `gold_despesa_analitica`, construída na camada Gold Flat.

Table											
	1º ID DESPESA	1º ID CANDIDATO	1º CANDIDATO	1º PARTIDO	1º CARGO	1º ID ESTADO	1º ESTADO	1º TOTAL ELEITORES	1º ID CATEGORIA DESPESA	1º CATEGORIA DESPESA	
1	74870484	70002550992	NATANRY LUDOVICO LACERDA OSORIO	PSDB	DEPUTADO DISTRITAL	8	DF	2253238	107862	Despesas com pessoal	
2	74870487	70002550992	NATANRY LUDOVICO LACERDA OSORIO	PSDB	DEPUTADO DISTRITAL	8	DF	2253238	265728	Publicidade por materiais impr	
3	74870483	70002550992	NATANRY LUDOVICO LACERDA OSORIO	PSDB	DEPUTADO DISTRITAL	8	DF	2253238	317609	Serviços prestados por tercei	
4	74870486	70002550992	NATANRY LUDOVICO LACERDA OSORIO	PSDB	DEPUTADO DISTRITAL	8	DF	2253238	45507	Atividades de militância e mo	
5	74870488	70002550992	NATANRY LUDOVICO LACERDA OSORIO	PSDB	DEPUTADO DISTRITAL	8	DF	2253238	129101	Encargos financeiros, taxas bar	
6	74870482	70002550992	NATANRY LUDOVICO LACERDA OSORIO	PSDB	DEPUTADO DISTRITAL	8	DF	2253238	122250	Encargos financeiros, taxas bar	
7	74870485	70002550992	NATANRY LUDOVICO LACERDA OSORIO	PSDB	DEPUTADO DISTRITAL	8	DF	2253238	122250	Encargos financeiros, taxas bar	
8	74870481	70002550992	NATANRY LUDOVICO LACERDA OSORIO	PSDB	DEPUTADO DISTRITAL	8	DF	2253238	310813	Serviços prestados por tercei	
9	74870484	70002550992	NATANRY LUDOVICO LACERDA OSORIO	PSDB	DEPUTADO DISTRITAL	8	DF	2253238	107839	Despesas com pessoal	
10											
10 rows 15.407s runtime											
Refreshed 3h ago											

A utilização dessa estrutura permite realizar as consultas de negócio em uma única tabela, reunindo as informações de despesas, candidatos, estados, categorias e período necessárias para responder aos questionamentos definidos no início do projeto.

As análises foram realizadas utilizando Spark SQL no ambiente Databricks. Para cada pergunta de negócio, foi executada uma consulta específica, seguida da apresentação dos resultados e de uma interpretação dos principais aspectos observados.

Pergunta 1

Quais são os 5 estados com maior valor total de despesas eleitorais contratadas em 2026?

12:26 PM (100)

110

Tsql

```
SELECT
  ESTADO,
  SUM(VALOR_DESPESA) AS TOTAL_DESPESAS
FROM gold_despesa_analitica
WHERE ESTADO <> 'BR'
GROUP BY ESTADO
ORDER BY TOTAL_DESPESAS DESC
LIMIT 5;
```

See performance (1)

Checked for improvements

Table

+

Q

	ESTADO	TOTAL_DESPESAS
1	SP	505971229.52
2	RJ	278526812.31
3	MG	266345816.70
4	BA	215210081.12
5	PR	182009245.65

5 rows

9.799s runtime

Refreshed 3h ago

This result is stored as `_sqlid#` and can be used in other `Python` and `SQL` cells.

Análise

Os resultados mostram que São Paulo apresenta o maior valor total de despesas eleitorais contratadas, com **R\$ 505,97 milhões**, seguido por Rio de Janeiro (**R\$ 278,53 milhões**), Minas Gerais (**R\$ 266,35 milhões**), Bahia (**R\$ 215,21 milhões**) e Paraná (**R\$ 182,01 milhões**).

Considerando apenas os cinco estados apresentados, o total de despesas é de aproximadamente **R\$ 1,45 bilhão**. **São Paulo concentra cerca de 34,9%** desse valor e **registra um volume aproximadamente 1,8 vez maior que o Rio de Janeiro, segundo colocado**. Os resultados evidenciam uma concentração relevante das despesas entre as maiores unidades federativas, mas com certa disparidade para São Paulo.

Pergunta 1.1

Quais são os 3 candidatos com maior valor total de despesas contratadas em cada um desses 5 estados?

```

--sql
WITH top5_estados AS (
  SELECT
    ESTADO
  FROM gold_despesa_analitica
  WHERE ESTADO <> 'BR'
  GROUP BY ESTADO
  ORDER BY SUM(VALOR_DESPESA) DESC
  LIMIT 5
),

despesas_por_candidato AS (
  SELECT
    g.ESTADO,
    g.ID_CANDIDATO,
    g.CANDIDATO,
    g.PARTIDO,
    g.CARGO,
    SUM(g.VALOR_DESPESA) AS TOTAL_DESPESAS_CANDIDATO
  FROM gold_despesa_analitica g
  INNER JOIN top5_estados t
    ON g.ESTADO = t.ESTADO
  GROUP BY
    g.ESTADO,
    g.ID_CANDIDATO,
    g.CANDIDATO,
    g.PARTIDO,
    g.CARGO
),

ranking AS (
  SELECT
    *,
    ROW_NUMBER() OVER (
      PARTITION BY ESTADO
      ORDER BY TOTAL_DESPESAS_CANDIDATO DESC
    ) AS RANK
  FROM despesas_por_candidato
)

SELECT
  ESTADO,
  RANK,
  CANDIDATO,
  PARTIDO,
  CARGO,
  TOTAL_DESPESAS_CANDIDATO
FROM ranking
WHERE RANK <= 3
ORDER BY ESTADO, RANK;
```

ESTADO	RANK	CANDIDATO	PARTIDO	CARGO	TOTAL DESPESAS CANDIDATO
BA	1	ANTONIO CARLOS PEIXOTO DE MAGALHÃES NETO	UNIÃO	GOVERNADOR	12315031.40
BA	2	JERONIMO RODRIGUES SOUZA	PT	GOVERNADOR	6645210.66
BA	3	JOÃO INACIO RIBEIRO ROMA NETO	PL	SENADOR	4842635.70
MG	1	ALEXANDRE KALIL	PDT	GOVERNADOR	13543286.38
MG	2	MATEUS SIMÕES DE ALMEIDA	PSD	GOVERNADOR	8601293.51
MG	3	PATRIUS ANANIAS DE SOUSA	PT	GOVERNADOR	7144089.25
PR	1	SERGIO FERNANDO MORO	PL	GOVERNADOR	7893971.92
PR	2	SANDRO ALEX CRUZ DE OLIVEIRA	PSD	GOVERNADOR	7259251.11
PR	3	DELTAN MARTINAZZO DALLAGNOL	NOVO	SENADOR	3795383.87
RJ	1	DOUGLAS RUAS DIOS SANTOS	PL	GOVERNADOR	12417923.82
RJ	2	EDUARDO DA COSTA PAES	PSD	GOVERNADOR	7692232.28
RJ	3	BENEDITA SOUZA DA SILVA SAMPAIO	PT	SENADOR	4608473.73
SP	1	TARCISIO GOMES DE FREITAS	REPUBLICANOS	GOVERNADOR	23999587.10
SP	2	FERNANDO HADDAD	PT	GOVERNADOR	12281657.76
SP	3	ANDRE LUIS DO PRADO	PL	SENADOR	4183436.50

Análise

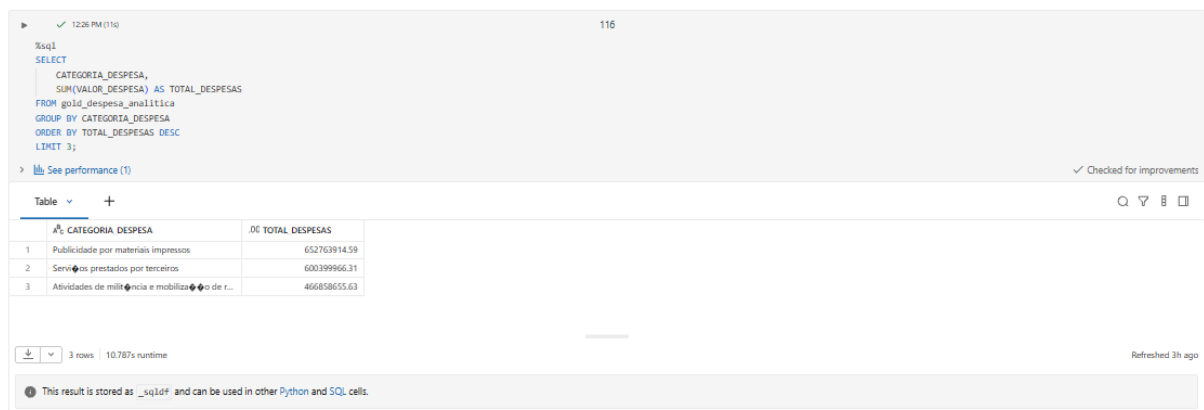
Os resultados mostram diferenças na distribuição das despesas entre os três candidatos com maiores valores em cada estado. Em **São Paulo**, o primeiro colocado concentra aproximadamente **59,3%** do valor total dos três candidatos analisados, apresentando uma diferença significativa em

relação ao segundo e ao terceiro colocados. Em **Bahia e Rio de Janeiro**, o primeiro candidato também representa cerca de metade das despesas dos três primeiros.

Já em **Paraná**, os valores dos dois primeiros candidatos são bastante próximos, enquanto em **Minas Gerais** observa-se uma distribuição mais equilibrada entre os três primeiros colocados. A análise permite, assim, observar como as despesas contratadas se distribuem entre os principais candidatos de cada estado - concentrando os principais valores por estado entre as candidaturas dos seus respectivos candidatos ao cargo de governador.

Pergunta 2

Quais são as 3 principais origens das despesas eleitorais contratadas em 2026?



The screenshot shows a Jupyter Notebook interface. At the top, there's a code cell with an SQL query. Below it, a table widget displays the results of the query. The table has two columns: 'CATEGORIA DESPESA' and '% TOTAL DESPESAS'. It contains three rows of data, representing the top three categories of electoral expenses.

```
%sql
SELECT
    CATEGORIA_DESPESA,
    SUM(VALOR_DESPESA) AS TOTAL_DESPESAS
FROM gold_despesa_analitica
GROUP BY CATEGORIA_DESPESA
ORDER BY TOTAL_DESPESAS DESC
LIMIT 3;
```

	CATEGORIA DESPESA	% TOTAL DESPESAS
1	Publicidade por materiais impressos	652763914.59
2	Serviços prestados por terceiros	600399966.31
3	Atividades de militância e mobilização de r...	466858655.63

3 rows | 10.787s runtime | Refreshed 3h ago

This result is stored as _sqlid# and can be used in other Python and SQL cells.

Análise

Entre as três principais origens de despesas, **Publicidade por materiais impressos** apresenta o maior valor, com aproximadamente **R\$ 652,76 milhões**, seguida por **Serviços prestados por terceiros**, com **R\$ 600,40 milhões**. Os valores das duas categorias são próximos, com uma diferença de aproximadamente **R\$ 52,36 milhões**.

Já **Atividades de militância e mobilização de rua** totalizam aproximadamente **R\$ 46,69 milhões**, valor consideravelmente inferior aos dois primeiros. Somadas, as duas principais categorias representam cerca de **96,4% do total das três maiores origens**, indicando forte concentração das despesas que configuram propaganda direta, visando impactar os eleitores.

Pergunta 2.1

Quais são as 3 principais descrições de despesas dentro de cada uma das 3 principais origens?

```

--sql
WITH top3_origens AS (
  SELECT
    CATEGORIA_DESPESA
  FROM gold_despesa_analitica
  GROUP BY CATEGORIA_DESPESA
  ORDER BY SUM(VALOR_DESPESA) DESC
  LIMIT 3
),
despesas_por_origem AS (
  SELECT
    g.CATEGORIA_DESPESA,
    g.DESPESA_DETALHADA,
    SUM(g.VALOR_DESPESA) AS TOTAL_DESPESAS
  FROM gold_despesa_analitica g
  INNER JOIN top3_origens t
    ON g.CATEGORIA_DESPESA = t.CATEGORIA_DESPESA
  GROUP BY
    g.CATEGORIA_DESPESA,
    g.DESPESA_DETALHADA
),
ranking AS (
  SELECT
    *,
    ROW_NUMBER() OVER (
      PARTITION BY CATEGORIA_DESPESA
      ORDER BY TOTAL_DESPESAS DESC
    ) AS RANK
  FROM despesas_por_origem
)

SELECT
  CATEGORIA_DESPESA,
  RANK,
  DESPESA_DETALHADA,
  TOTAL_DESPESAS
FROM ranking
WHERE RANK <= 3
ORDER BY CATEGORIA_DESPESA, RANK;
```

	A ₀ CATEGORIA_DESPESA	R ₀ RANK	A ₀ DESPESA_DETALHADA	.00% TOTAL_DESPESAS
1	Atividades de militância e mobilização de rua	1	CABO ELEITORAL	19542479.81
2	Atividades de militância e mobilização de rua	2	PANFLETAGEM	7733243.44
3	Atividades de militância e mobilização de rua	3	MILITANCIA	7354953.96
4	Publicidade por materiais impressos	1	SANTINHOS	4469361.12
5	Publicidade por materiais impressos	2	WIND BANNER	2953966.55
6	Publicidade por materiais impressos	3	WINDBANNER	1950583.09
7	Serviços prestados por terceiros	1	SERVICOS ESPECIALIZADOS DE COMUNICACAO MARKETING p - 920125	31000000.00
8	Serviços prestados por terceiros	2	SERVICOS DE PLANEJAMENTO E MARKETING POLITICO, CONFORME NF 108 E 11...	13100000.00
9	Serviços prestados por terceiros	3	Serviços de Marketing Digital	10000000.00

Análise

Dentro das três principais origens de despesas, observa-se que cada categoria apresenta uma concentração em determinados tipos de gasto. Em **Atividades de militância e mobilização de rua**, a maior despesa é **Cabo Eleitoral**, com aproximadamente **R\$ 19,54 milhões**, seguida por **Panfletagem (R\$ 7,73 milhões)** e **Militância (R\$ 7,35 milhões)**.

Em **Publicidade por materiais impressos**, destacam-se **Santinhos**, com aproximadamente **R\$ 44,69 milhões**, seguidos por **Wind Banner (R\$ 2,95 milhões)** e **Windbanner (R\$ 1,95 milhão)**.

Já em **Serviços prestados por terceiros**, os maiores valores estão concentrados em **Serviços especializados de comunicação e marketing (R\$ 31,0 milhões)**, **Serviços de planejamento e marketing político (R\$ 13,1 milhões)** e **Serviços de Marketing Digital (R\$ 10,0 milhões)**.

Os resultados mostram quais tipos específicos de despesas possuem maior participação dentro de cada uma das principais categorias.

Pergunta 3

Qual é o custo das despesas eleitorais contratadas, por eleitor, em cada estado brasileiro em 2026?

12:26 PM (17s)

122

```

Tsql
WITH estado AS (
    SELECT
        ESTADO,
        SUM(VLOR_DESPESA) AS TOTAL_DESPESAS,
        MAX(TOTAL_ELEITORES) AS TOTAL_ELEITORES
    FROM gold_despesa_analitica
    WHERE ESTADO <> 'RR'
    GROUP BY ESTADO
)

SELECT
    ESTADO,
    TOTAL_DESPESAS,
    TOTAL_ELEITORES,
    ROUND(
        TOTAL_DESPESAS / NULLIF(TOTAL_ELEITORES, 0),
        2
    ) AS CUSTO_POR_ELEITOR
FROM estado
ORDER BY CUSTO_POR_ELEITOR DESC;

```

See performance (1)

Checked for improvements

Table

	ESTADO	TOTAL DESPESAS	TOTAL ELEITORES	CUSTO POR ELEITOR
1	RR	69268266.03	401573	172.49
2	AP	63725155.00	577698	110.31
3	AC	42829163.78	614631	69.68
4	RO	83402858.75	1268667	65.74
5	TO	70694343.41	1182768	59.77
6	SE	89734800.50	1740240	51.56
7	MT	103530781.01	2639075	39.23
8	DF	84835489.00	2253238	37.65
9	MS	72087040.12	2025001	35.60
10	PI	92227284.76	2708756	34.05
11	PB	103702624.52	3248007	31.93
12	RN	84040518.82	2660947	31.58
13	AM	78126954.47	2802091	27.88
14	AL	66669793.94	2442275	27.30
15	MA	135670540.04	5188115	26.15

27 rows | 16.603s runtime

Refreshed 3h ago

Análise

Os resultados mostram diferenças relevantes no custo das despesas eleitorais contratadas por eleitor entre os estados. **Roraima (RR)** apresenta o maior valor, com **R\$ 172,49 por eleitor**, seguido por **Amapá (R\$ 110,31)** e **Acre (R\$ 69,68)**. Na sequência aparecem **Rondônia (R\$ 65,74)** e **Tocantins (R\$ 59,77)**.

Essa diferença mostra que, em alguns estados, há uma **maior intensidade de recursos contratados em relação ao tamanho do eleitorado**. Roraima, por exemplo, apresenta um custo por eleitor aproximadamente **56% superior ao Amapá**.

Esse indicador pode servir como ponto de partida para investigar por que determinadas unidades federativas concentram mais recursos por eleitor, embora, isoladamente, não permita concluir que maiores gastos resultem em maior alcance ou impacto sobre os eleitores.

7. Discussão Geral dos Resultados

A análise conjunta das perguntas de negócio evidencia que o principal aspecto observado não está apenas no volume das despesas eleitorais, mas na forma como esses recursos se distribuem entre estados, candidatos e categorias de gasto. **Os resultados demonstram diferenças relevantes no volume total contratado e, principalmente, no custo das despesas em relação ao tamanho do eleitorado de cada estado.**

Esse cenário revela uma forte assimetria na aplicação dos recursos, com concentração das despesas em determinadas unidades federativas, candidatos e categorias. Embora os dados não permitam avaliar a efetividade ou o impacto desses gastos sobre os eleitores, as diferenças observadas indicam um ponto relevante para investigação: compreender quais fatores explicam essas disparidades e como o volume de recursos contratados se relaciona com a escala de cada eleitorado.

Esse levantamento também abre espaço para questionamentos relacionados ao acompanhamento, à transparência e à distribuição dos recursos empregados no processo eleitoral, especialmente diante das diferenças observadas entre as unidades federativas e entre as categorias de despesas.

De forma geral, o trabalho permitiu responder aos questionamentos definidos no início do projeto, consolidando as informações em uma estrutura analítica que facilita a consulta e a interpretação dos dados.

8. Pontos de Melhoria

Uma das principais oportunidades de evolução do projeto seria o tratamento mais aprofundado dos registros incompletos ou anômalos identificados nas bases de origem. Embora esses registros tenham sido preservados na camada Silver para manter a fidelidade aos dados disponibilizados pelo TSE, uma etapa adicional de tratamento poderia classificar, corrigir ou excluir registros inadequados antes da consolidação da camada Gold, aumentando a precisão das métricas e análises realizadas.

Outra melhoria seria a automatização da atualização das bases. Devido ao volume dos arquivos e às limitações do ambiente Databricks utilizado no projeto, a coleta e o processamento foram realizados manualmente diretamente no Databricks, com os códigos disponibilizados no GitHub. Em uma implementação produtiva, seria desejável automatizar a ingestão e o processamento periódico dos dados, garantindo maior atualização e reprodutibilidade do pipeline, principalmente em períodos eleitorais, nos quais informações atualizadas podem contribuir para análises e acompanhamento do processo eleitoral.

Por fim, poderia ser implementado um processo de monitoramento da qualidade dos dados a cada nova carga. Indicadores como quantidade de nulos, registros anômalos, chaves sem correspondência, duplicidades e variações no volume das bases poderiam ser verificados automaticamente, permitindo identificar alterações ou problemas na fonte antes que afetem as análises.

9. Autoavaliação

Em um âmbito pessoal, este foi um projeto extremamente enriquecedor e prazeroso de realizar. O módulo foi uma introdução a um nível de programação com o qual eu ainda não estava acostumado, mas fiquei muito feliz por ter tido a oportunidade de praticar, aprender e desenvolver análises que, no início do módulo, eu não imaginava ser capaz de construir.

Independentemente do resultado final, estou muito feliz com a minha entrega e, principalmente, com o meu desenvolvimento ao longo do projeto.

Consegui escolher um tema às vésperas de um momento importante, trabalhar com dados reais e construir uma análise que pode ser útil para pessoas interessadas no assunto. Provavelmente em alguns meses, vou enxergar diversas coisas que poderiam ser melhoradas, mas acredito que isso faz parte do processo de adaptação a uma nova área.

Saio deste módulo mais confiante nos meus conhecimentos e ainda mais entusiasmado para continuar aprendendo e me desenvolvendo com um próximo desafio.